

Clase 5:

Estructuras de Control : Repetición – Ciclos Exactos

Hasta el momento, hemos estudiado dos categorías fundamentales dentro de las estructuras de control:

Estructuras de decisión: (como if e if-else), que permiten bifurcar el flujo de ejecución según se evalúe una condición booleana.

Ciclos inexactos: (como el ciclo while), que repiten un bloque de código mientras una condición se cumpla, sin que necesariamente podamos determinar a priori cuántas iteraciones ocurrirán.

En los ciclos inexactos, el número de repeticiones depende de la evolución en tiempo de ejecución de una condición. Esta evolución puede estar ligada a valores ingresados por el usuario, resultados de cálculos intermedios, o bien a eventos externos no predecibles antes de ejecutar el programa.

En esta clase nos centraremos en el segundo gran grupo de ciclos: los **ciclos exactos**.

Abordaremos los siguientes puntos:

Definición formal y características.

Comparación con los ciclos inexactos.

Problemas típicos que se resuelven con ciclos exactos.

Implementación en C++ mediante la estructura **for**, analizando su sintaxis y funcionamiento.

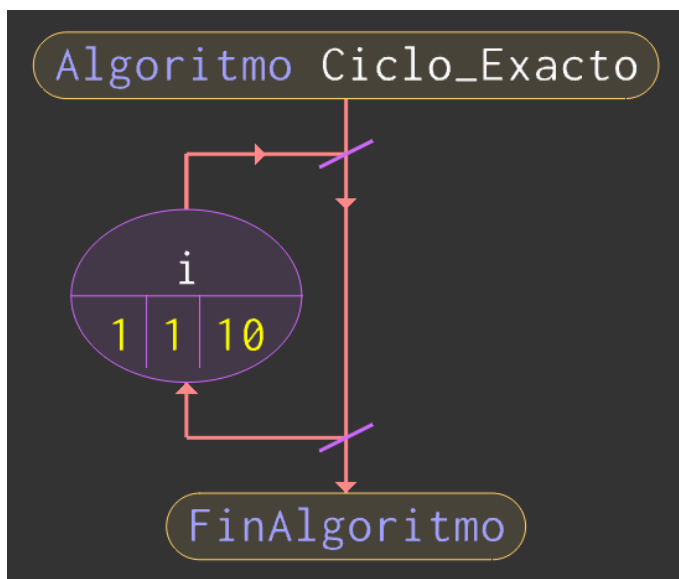
Ciclo Exacto:

Un **ciclo exacto** es un bucle en el que **sabemos de antemano cuántas veces se va a repetir**.

Es decir, antes de que empiece a ejecutarse, ya podemos calcular la cantidad de iteraciones mediante una regla matemática o lógica, sin depender de lo que vaya pasando durante la ejecución.

Para que un ciclo sea exacto, deben estar claramente definidos los siguientes elementos:

1. **Variable de control:** un identificador que toma distintos valores a lo largo de las iteraciones.
2. **Valor inicial:** el primer valor que toma la variable de control.
3. **Condición de ejecución (o finalización):** una expresión booleana que determina cuándo detener el ciclo.
4. **Regla de actualización:** un incremento o decremento (generalmente constante) que se aplica a la variable de control al final de cada iteración.



Como puede observarse en las imágenes, la estructura de un ciclo exacto se compone de los siguientes elementos fundamentales:

Variable de control: Se define una variable que permitirá controlar el número de iteraciones del ciclo. En este caso, la variable es *i*. Esta variable será la encargada de determinar cuántas veces se ejecutará el bloque de instrucciones.

Valor inicial: Se establece el valor a partir del cual comenzará la variable de control. En el ejemplo, *i* inicia en 1. Este valor marca el punto de partida del ciclo.

Condición de ejecución: Se define la condición lógica que debe cumplirse para que el ciclo continúe ejecutándose. En este caso, el ciclo se repetirá mientras *i* sea menor o igual a 10. Cuando esta condición deje de cumplirse, el ciclo finalizará.

Regla de actualización: Se determina cómo se modificará la variable de control al finalizar cada iteración. En este ejemplo, *i* se incrementa en una unidad (*i++*) al terminar cada vuelta, lo que garantiza que el ciclo avance progresivamente hacia su condición de finalización.

En conjunto, estos cuatro componentes permiten que el ciclo exacto tenga un comportamiento ordenado, predecible y matemáticamente determinable.

```
int main() {  
    int i;  
  
    for(i=1;i<=10;i++){  
        //INSTRUCCIONES  
    }  
  
    return 0;  
}
```

Características y uso del ciclo exacto

Los ciclos exactos poseen las siguientes características:

- **Predictibilidad:** el número de repeticiones es independiente de la entrada del usuario o de condiciones cambiantes durante la ejecución.
- **Control centralizado:** la variable de control es gestionada directamente por la estructura del ciclo.
- **Adecuación a secuencias acotadas:** se utiliza cuando el problema implica recorrer un intervalo numérico conocido, una cantidad fija de elementos, o cualquier situación con límites bien definidos.

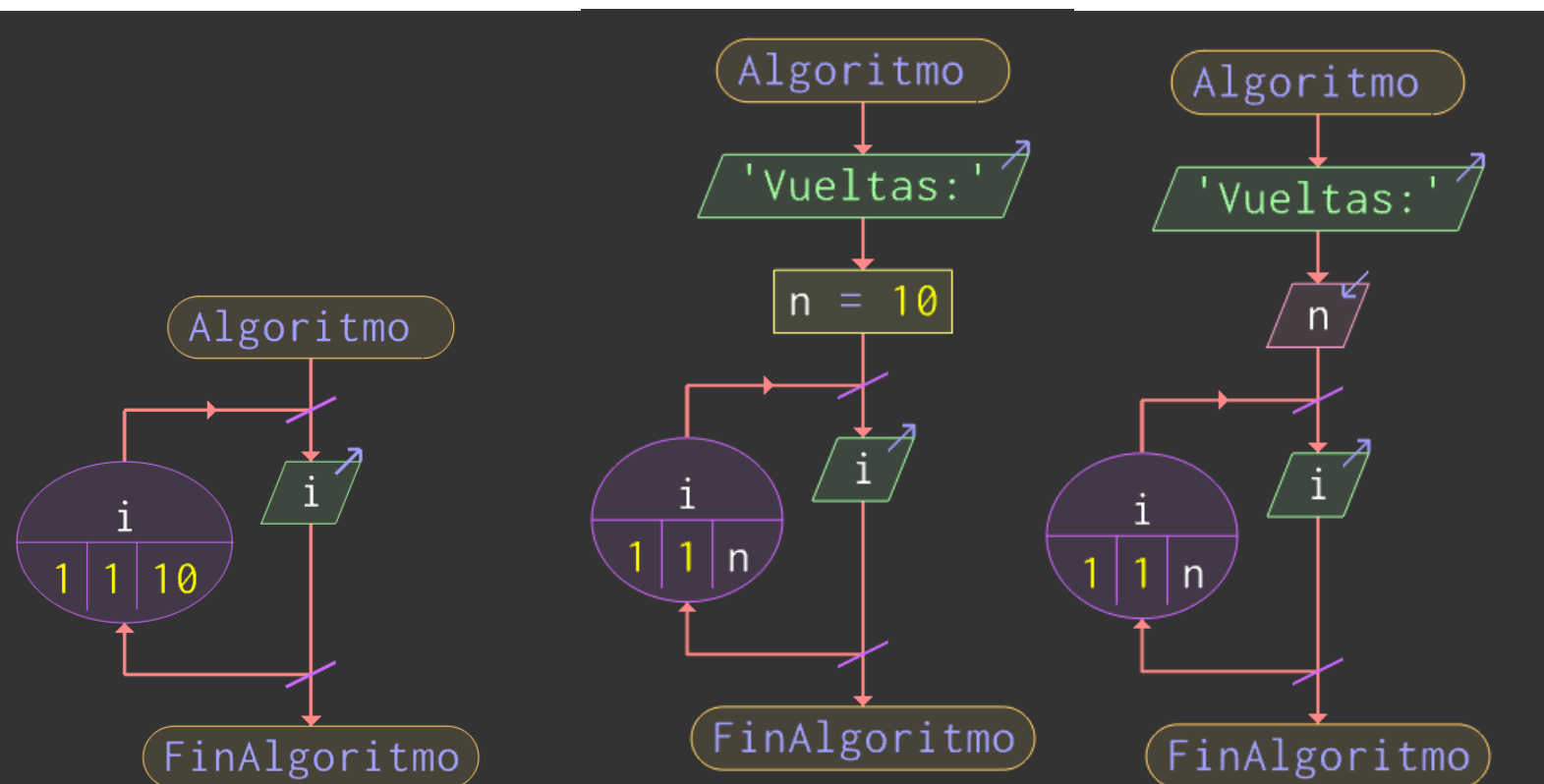
¿Cuándo corresponde usar un ciclo exacto?

Un ciclo exacto es la elección natural cuando la cantidad de iteraciones está completamente determinada antes de que el ciclo comience a ejecutarse. Esta determinación puede provenir de dos situaciones distintas, pero igualmente válidas desde el punto de vista del ciclo exacto:

- **Valores concretos fijos:** Constantes literales, como 5, 10, 100, conocidos en tiempo de compilación.
- **Valores almacenados en variables:** Valores que se calculan o ingresan durante la ejecución, pero que ya están disponibles antes de iniciar el ciclo.

Es importante destacar que la "exactitud" del ciclo no exige que el número de iteraciones sea una constante escrita en el código fuente. Basta con que, en el momento de llegar al ciclo, el programa ya sepa cuántas iteraciones realizará, ya sea porque ese número se obtuvo de una entrada del usuario, de un cálculo previo o de una constante.

Algunos ejemplos:



¿Cuándo conviene utilizar cada tipo de ciclo?

En programación, la elección entre un ciclo exacto y un ciclo inexacto no es arbitraria. Depende directamente del tipo de problema que se desea resolver y, principalmente, de si la cantidad de repeticiones está determinada o no antes de comenzar la ejecución.

Un **ciclo inexacto** se utiliza cuando necesitamos que un conjunto de instrucciones se ejecute una cantidad **no especificada de veces**, es decir, cuando el número de iteraciones depende de una condición cuyo cumplimiento no puede determinarse previamente.

En estos casos, el ciclo continúa ejecutándose mientras la condición sea verdadera, pero no sabemos con certeza cuántas repeticiones habrá.

Ejemplo:

- Ingresar números hasta que el usuario escriba un número negativo.
- Solicitar contraseñas hasta que la ingresada sea correcta.
- Leer datos hasta que se detecte un valor específico (por ejemplo, 0).

En todos estos ejemplos, la repetición depende de una decisión externa (generalmente del usuario) y no es posible calcular anticipadamente cuántas veces se ejecutará el ciclo.

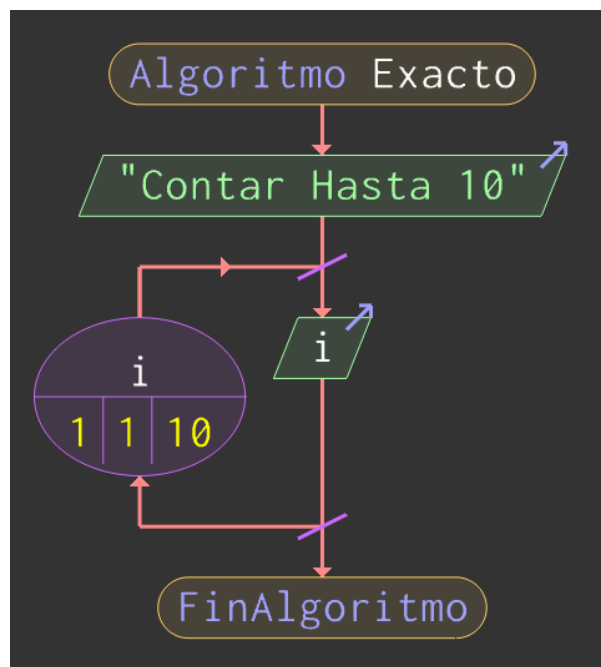
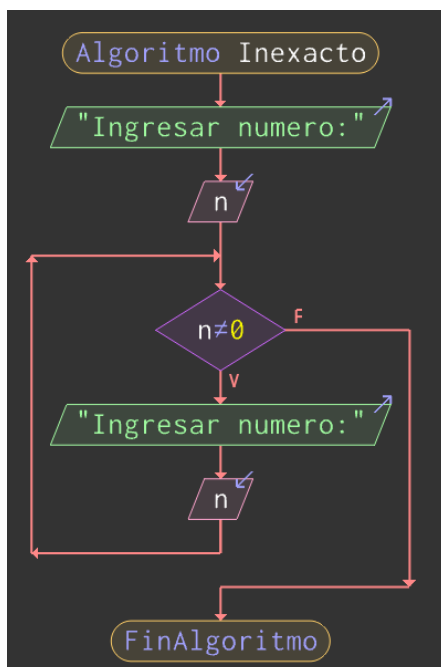
Un **ciclo exacto** se utiliza cuando queremos repetir un conjunto de instrucciones una cantidad **determinada y conocida de veces**.

La cantidad de iteraciones está establecida antes de que el ciclo comience y puede calcularse en función de un valor inicial, un valor final y una regla de actualización.

Ejemplo:

- Ingresar 3 notas de un alumno.
- Registrar 5 temperaturas.
- Procesar 10 ventas.
- Mostrar los números del 1 al 100.

En estos casos, el problema establece explícitamente cuántas repeticiones deben realizarse, por lo que resulta apropiado utilizar un ciclo exacto.



Estructura for en C++

El ciclo for es la estructura más utilizada en C++ para implementar **ciclos exactos**, ya que integra en una sola línea los tres elementos fundamentales de este tipo de repetición: inicialización, condición y actualización.

Sintaxis General

La forma general del ciclo for es la siguiente:

```
for(inicializacion; condicion; actualizacion){  
    // bloque de instrucciones  
}
```

Por ejemplo:

```
int main() {  
    //EJEMPLO: Contar hasta 10  
    for(int i=1;i<=10;i++){  
        cout<<i<<endl;  
    }  
    return 0;  
}
```

Como vemos en el ciclo:

En la primera parte del for se declara e inicializa la variable de control; en este caso, `int i = 1`. Esta variable será la encargada de controlar la cantidad de iteraciones del ciclo y comienza con el valor 1.

En la segunda parte se establece la **condición**; en este ejemplo, `i <= 10`. Esto significa que el ciclo se ejecutará **mientras** `i` sea menor o igual a 10. Cada vez que se vaya a iniciar una nueva vuelta, esta condición será evaluada. Si resulta verdadera, el ciclo continúa; si resulta falsa, el ciclo finaliza.

Por último, en la tercera parte se define la **actualización** de la variable de control; en este caso, `i++`, lo que indica que al finalizar cada iteración el valor de `i` se incrementa en una unidad. Esta actualización es la que permite que la variable avance progresivamente hacia el valor límite establecido en la condición.

De esta manera, la estructura del for concentra en una sola línea el inicio, el control y la progresión del ciclo, garantizando un comportamiento ordenado y predecible.